Rapport Technique : Pipeline CI/CD

Automatisation DevOps avec GitHub Actions & Docker Hub

> **Objectif :** Ce document détaille la mise en place d'une chaîne d'intégration et de déploiement continu (CI/CD) automatisée pour une application Node.js, incluant le test, la conteneurisation et la publication sur Docker Hub.

1. Architecture de la Pipeline

La pipeline est définie dans le fichier `.github/workflows/main.yml` et se compose de trois étapes principales interdépendantes :

- **Test :** Validation du code source via Node.js.
- **Build-and-Push :** Création de l'image Docker et envoi vers le registre distant.
- **Deploy :** Simulation du déploiement en production.

2. Analyse des Problèmes et Résolution

2.1 Échec Initial de l'Authentification

Lors de la première exécution, l'étape `Login to Docker Hub` a échoué avec l'erreur suivante :

```
ERROR: USERNAME AND PASSWORD REQUIRED
```

Cause : Les identifiants n'étaient pas fournis à l'action `docker/login-action@v3`, empêchant la connexion au registre Docker Hub.

2.2 Solution : Gestion des Secrets GitHub

Pour sécuriser les accès sans exposer les mots de passe dans le code source, nous avons utilisé les **GitHub Secrets**.

Secret	Description / Valeur
`DOCKERHUB_USERNAME`	Nom d'utilisateur Docker Hub (haddouanas16404)
`DOCKERHUB_TOKEN`	Personal Access Token (PAT) généré pour l'authentification sécurisée

3. Configuration du Workflow (YAML)

Voici l'extrait critique permettant une authentification sécurisée et un build automatisé :

```
- name: Login to Docker Hub
  uses: docker/login-action@v3
  with:
    username: ${{ secrets.DOCKERHUB_USERNAME }}
    password: ${{ secrets.DOCKERHUB_TOKEN }}

- name: Build and push
  uses: docker/build-push-action@v5
  with:
    context: .
    push: true
    tags: ${{ secrets.DOCKERHUB_USERNAME }}/mini-devops-project:latest
    build-args: |
      APP_VERSION=1.1.0
      COMMIT_HASH=${{ github.sha }}
```

4. Résultats et Validation

STATUT : SUCCÈS (BUILD #3)

Après configuration des secrets et mise à jour du script, la pipeline s'exécute avec succès en moins de 60 secondes :

- **Test (10s) :** `npm test` validé.
- **Build-and-Push (25s) :** Image Docker compilée et poussée sur Docker Hub.
- **Deploy (2s) :** Déclenchement de la procédure de déploiement.

Conclusion

La mise en place de cette pipeline démontre une maîtrise des outils DevOps modernes. L'utilisation des **Secrets** garantit une sécurité optimale, tandis que l'automatisation permet de réduire les erreurs humaines lors des mises à jour applicatives.